🏀 NBA Breakout Player Predictor

End-to-end ML pipeline identifying NBA players poised for a breakout season

🔗 nba-breakout-predictor.streamlit.app 💻 github.com/ChenEylon/NBA-breakout-predictor

Python 3.10+ | LightGBM | XGBoost | scikit-learn | SHAP | Streamlit | Plotly | SQLite | nba_api

PROJECT OVERVIEW

This project answers a concrete sports analytics question: **which NBA players are most likely to have a breakout season next year?** A breakout is defined as a ≥20% improvement in Points, PIE (Player Impact Estimate), or True Shooting % versus the prior season, subject to minimum playing time thresholds (≥40 GP, ≥20 MPG).

The system ingests 15 years of NBA statistics, engineers 60+ predictive features, trains and compares three ensemble models, explains predictions using SHAP values, and serves everything through an interactive Streamlit dashboard — deployed publicly on Streamlit Cloud.

PIPELINE ARCHITECTURE

Phase 1 Data Ingestion → Phase 2 Feature Engineering → Phase 3 Model Training → Phase 4 SHAP Explainability → Phase 5 Streamlit Dashboard

Phase 1 — Data Acquisition

- 15 seasons (2010-11 → 2025-26) from `nba_api`
- Base stats: PTS, AST, REB, STL, BLK, TOV, FG%/3P%/FT%
- Advanced stats: TS%, USG%, PIE, EFG%, Net/Off/Def Rating
- SQLite storage with JSON caching + retry logic

Phase 2 — Feature Engineering

- Lag features: prior-season values for all 16 stats
- YoY deltas and % changes (trajectory signals)
- Demographics: age, years in league, team change
- No data leakage — only prior-season stats as features

Phase 3 — Model Training

- Train: 2010-25 (15 seasons) | Val: 2025-26 (held out)
- XGBoost, Random Forest, LightGBM
- Class imbalance handling (~14% breakout rate)
- Threshold optimized for precision via CV OOF probabilities

Phase 4 & 5 — Explainability + Dashboard

- SHAP TreeExplainer — top 8 drivers per player
- 3 dashboard views: Top 20, Player Deep Dive, Model Performance
- Radar charts, SHAP bar charts, PR curves (Plotly)
- Sidebar filters: min probability, age range

MODEL RESULTS

Evaluated on the 2025-26 held-out validation season. **LightGBM selected** as the best model — precision was prioritized since surfacing a short, high-confidence list is more actionable than a long list with many false positives.

Model	Precision	Recall	F1	AUC-ROC	Threshold
⭐ **LightGBM**	**0.563**	**0.243**	**0.340**	**0.655**	**0.717**
Random Forest	0.550	0.297	0.386	0.717	0.662
XGBoost	0.500	0.162	0.245	0.677	0.722

> **Baseline comparison:** A naive classifier predicting every player as "no breakout" would achieve 0% precision on breakouts. The random baseline for precision on this dataset is ~14% (the breakout rate). LightGBM's 56% precision is 4× above baseline, identifying real signal in player trajectory.

15	60+	3	56%
Seasons of training data	Engineered features	Models compared	Best precision (4× baseline)

KEY DESIGN DECISIONS

No data leakage

Features are strictly prior-season statistics. Current-season stats define the label but are never used as inputs — a common pitfall in sports ML projects.

Threshold optimization

Default 0.5 thresholds are rarely optimal for imbalanced datasets. Each model's cutoff is tuned on OOF predictions to maximize precision at ≥15% recall.

Caching at every stage

API responses → JSON cache, features → Parquet, models → joblib. Full pipeline runs once; dashboard reads pre-built artifacts for instant load.

SHAP for transparency

TreeExplainer surfaces the specific features driving each prediction. This moves beyond a black-box score to actionable scouting intelligence.

LIMITATIONS & FUTURE WORK

- **Feature expansion:** Injury history, coaching changes, contract year status, and player tracking data (Second Spectrum) would likely improve precision significantly
- **Position data:** Not currently available from `LeagueDashPlayerStats`; enrichment via `CommonAllPlayers` would enable position-stratified analysis
- **Hyperparameter tuning:** Configs are hand-set; Optuna-based Bayesian optimization is a natural next step
- **Temporal cross-validation:** Walk-forward validation across multiple held-out seasons would give more robust performance estimates than a single held-out season

---

github.com/ChenEylon/NBA-breakout-predictor

nba-breakout-predictor.streamlit.app